Patrícia Cruz Azevedo

Busca binária

```
if(argc<2) fprintf(stderr,"uso"); return 1;
FILE *f = fopen(argv[2], "rb");
if(!f){
    fprintf(stderr,"erro");
    return 1;
}
fseek(f, 0, SEEK_END); int i,qt; Endereco e;
long numB = ftell(f);
long numR = numB/sizeof(Endereco);
long ini=0; long fim = numR-1; long meio; int achou=0;
while(fim >= ini){
    meio = (ini+fim)/2
    fseek(f, meio*sizeof(Endereco), SEEK_SET);
    qt=fread(&e, sizeof(Endereco), 1, f);
    long aux = strncmp(argv[1], e.cep, 8);
    if(aux ==0){
        //achou o cep
        achou=1;
        break;
    }
    else{
        if(aux<0)
            fim = meio-1;
        else ini = meio+1;
    }
}
if(achou==0)
    //cep não encontrado
fclose(f)
```

Qsort

```
FILE *f, *saida; Endereco *e;
long nos, qtd, metade;
//abrir, pegar numB e numR->qtd
e=(Endereco*)malloc(qtd*sizeof(Endereco));
rewind(f)
if(fread(e, sizeof(Endereco), qtd, f) == qtd)
//lido
qsort(e, qtd, sizeof(Endereco), compara);
//ordenado
saida=fopen("saida.dat", "wb");
fwrite(e, sizeof(Endereco), qtd, saida);
}
int compara(const void *e1, const void *e2){
    return strcmp(((Endereco*)e1)->cep, ((Endereco*)e2)->cep);
}
```

Intercala

```
FILE *a, *b; Endereco ea, eb
while(!feof(a) && !feof(b)){
    if(compara(&ea, &eb) < 0){
        fwrite(&ea, sizeof(Endereco), 1, saida);
        fread(&ea, sizeof(Endereco), 1, a);
    } else { //ea >= eb
        fwrite(&eb, sizeof(Endereco), 1, saida);
        fread(&eb, sizeof(Endereco), 1, b);
    }
}
while(!feof(a)){
    fwrite(&ea, sizeof(Endereco), 1, saida);
    fread(&ea, sizeof(Endereco), 1, a);
}
//mesmo de cima, mas troca por b
```

Índice

```
typedef struct _Indice Indice;
struct _Indice { char cep[8]; int ind; };
main{ //vetor da struct de Índices
    Indice* in = malloc(sizeof(Indice)*numR);
    if(in==null){ //erro retorna 1} int i=0;
    while(fread(&e, sizeof(Endereco), 1, f)==1){
        strcpy(in[i].cep, e.cep, 8);
        in[i].ind = i;
        i++;
    }
```

Árvore B guarda dados em todos os nós.
Árvore B+ guarda dados apenas nas folhas e interliga essas folhas com lista encadeada.
Busca por intervalo é mais rápido por conta disso. Ambas são O(Log n)
Inserção com estouro (split), a árvore joga a mediana para cima, enquanto a B+ joga pra cima mas mantem uma cópia na folha.

A remoção exige manter os nós com ocupação mínima de 50%. usando empréstimo de nós irmãos ou fusão quando necessário.

Empréstimo: Pai desce, filho sobe
Fusão: junta o nó com um irmão e o pai desce para unir os 2 em um único nó

Pseudo Código split

```
Struct Pag {
    int qt;
    int chaves[n];
    int valores[n];
    int filhos[n+1]
};
```

int = 4 bytes

4 + 4·n + 4·n + 4(n+1) = 12n+8 igualo isso ao valor requerido e descobre o valor de n.

```
void split(Pag *p1, Pag *p2, int novaChave, int novoValor, int* chave, int* valor){
    int meio = p1->qt/2;
    *chave = p1->chaves[meio];
    *valor = p1->valores[meio];
    int cont = 0;
    for(int i = meio+1; i < p1->qt; i++){
        p2->chaves[cont] = p1->chaves[i];
        p2->valores[cont] = p1->valores[i];
        p2->filhos[cont] = 0; //enunciado pede filhos nulos em folha
        cont++;
    }
    p2->chaves[cont] = novaChave; //enunciado garante ser o maior
    p2->valor[cont] = novoValor;
    p2->filhos[cont] = 0;
    p2->filhos[cont+1] = 0;
    p2->qt = cont+1;
    p1->qt = meio;
```

Hash → tabela hash é excelente para buscas de correspondência exata (saber um id específico). Tempo médio O(1), pior caso O(n). Péssimo em pesquisa de intervalos e ordenação, desorganiza a fórmula para garantir menos colisão. Encadeamento externo lista ligada em cada posição) ou encadeamento interno (procura próx espaço livre na tabela).

id	chave	valor	prox
0	32	1011	100
1	10	1111	∅
100	34	1101	∅

MAX 2048

```
srand(time(NULL));
float ranN = 1.0*rand() / RAND_MAX;
static char buffer[MAX]; char *prox = NULL;
fgets(buffer, MAX, f);
char * linha[15]; // Array com 15 ponteiros
strtok_r(buffer, ",\n", &prox); //virgula ou \n
linha[1] = strtok_r(NULL, ",\n", &prox);
```

```
void gerenciaIntercala(int k){
 char nomes[200][100];
 int totalArquivos = k;
 int atual = 0;
  for(int i = 0; i < k; i++)
     sprintf(nomes[i], "bloco%d.dat", i);
  while(totalArquivos - atual > 1){
     char *arq1 = nomes[atual++];
     char *arq2 = nomes[atual++];
     char nomeSaida[100];
     sprintf(nomeSaida, "temporario%d.dat", totalArquivos);
     strcpy(nomes[totalArquivos], nomeSaida);
     intercala(arq1, arq2, nomeSaida);
     remove(arq1);
     remove(arq2);
     totalArquivos++;
  }
printf("Arquivo final ordenado: %s\n", nomes[totalArquivos - 1]);
}

void processarBlocos(FILE *file, long totalBytes, int k){
    long totalElementos = totalBytes/sizeof(tipo);
    long elementosBloco = totalElementos/k;

    long tamanhoMax = elementosBloco + (totalElementos % k);
  //para caso seja um numero quebrado tipo 13/4, iria sobrar 1

    tipo *buffer = malloc(tamanhoMax*sizeof(tipo));

    long lidos;
    char nome[100];
    for(int i = 0; i < k; i++){
        if(i == k-1)
            lidos = totalElementos - (i*elementosBloco);
        else
            lidos = elementosBloco;

        fread(buffer, sizeof(tipo), lidos, file);
        qsort(buffer, lidos, sizeof(tipo), compara);
        sprintf(nome, "bloco%d.dat", i);

        FILE *temp = fopen(nome, "wb");
        if(temp){
            fwrite(buffer, sizeof(tipo), lidos, temp);
            fclose(temp);
        }
    }
    free(buffer);
}

int compara(const void *e1, const void *e2){
    if (*(tipo*)e1 < *(tipo*)e2)
        return -1;
    if (*(tipo*)e1 > *(tipo*)e2)
        return 1;
    return 0;
}
```